Turning LAMBDA into Python

A register-based compiler, using recursive factorial

CS413

Fall 2026

Compiler pipeline

1. **Parse source.** `d0exp_parse` turns LAMBDA text into expression nodes.
2. **Check types.** The type checker can reject invalid expressions before compilation.
3. **Compile the AST.** `d0exp_compile` lowers each node to registers and Python statements.
4. **Run Python.** The generated module defines `_lambda_main` and stores its result in `_lambda_result`.

Important interface detail

The compiler accepts an AST, not source text. A typical caller parses first, then passes the AST to `d0exp_compile`.

Input program: factorial

```
let
  fact : int -> int =
    fix fact(n: int): int =>
      if n <= 1 then 1 else n * fact(n - 1)
in
  (fact(0), fact(1), fact(5), fact(6))
end
```

- `fix` binds both the recursive function name and its parameter.
- The condition selects one branch; the other branch must not run.
- The final tuple makes several results visible in one evaluation.

The parser produces an AST

The outer expression is a sequential binding followed by a tuple result:

```
D0Elet(some name elided)
  value: D0Efix(fact, n, body)
    body: D0Eif0
      test: D0Eop2("<=", D0Evar(n), D0Eint(1))
      then: D0Eint(1)
      else: D0Eop2("*", D0Evar(n), recursive_call)
  body: D0Etupl(fact(0), fact(1), fact(5), fact(6))
```

`D0Eint`, `D0Efix`, and `D0Eif0` are dataclass variants. Types are annotations on AST nodes and are erased by code generation.

AST classes describe LAMBDA syntax

Expression variants inherit from `D0E000`; declarations inherit from `D0C000`. Their dataclass fields hold child nodes and source names.

```
D0Eint(value)
D0Evar(name)
D0Eop1(operator, operand)
D0Eop2(operator, left, right)
D0Eapp(function, argument)
D0Eif0(test, then, otherwise)
```

```
D0Elam(parameter, body, annotation)
D0Efix(name, parameter, body, ...)
D0Elet(name, value, body)
D0Elets(declarations, body)
D0Etupl(items)
D0Eproj(tuple, zero_based_index)
```

`D0Cval` represents a named declaration, including its optional type annotation and explicit type parameters. The aliases `d0exp` and `d0cls` name the expression and declaration base types.

Type classes represent checker data

```
TInt()        TBool()
TTuple(items)
TFun(argument, result)
TList(element) TOption(element)
TMeta(ident)  TPoly(name)
```

The variants form the `simple_type` union. The type dataclasses are frozen, so a type node is immutable after construction.

```
_Scheme(
  quantified={...},
  typ=TFun(TMeta(0), TMeta(0)))
```

`TMeta` holds an inference variable. `TPoly` marks a declared type parameter. `_Scheme` pairs a type with the inference variables quantified by a polymorphic declaration.

Runtime values and environments

The interpreter has its own class hierarchy; evaluating an AST produces `D0V` values rather than Python primitives.

```
D0Vint(number)   D0Vbtf(flag)
D0Vlist(items)  D0Voptn(option)
D0Vtupl(items)
D0Vlam(environment, lambda_ast)
D0Vfix(environment, fix_ast)
```

A function value keeps the syntax it will run and the environment needed for its free variables. This pair is a closure.

```
ENVnil()
ENVcns(name, value, rest)
```

Environments are linked bindings. Lookup checks the newest binding first, so an inner binding naturally shadows an outer one. The type `d0val` aliases the runtime value base class.

Classes supporting parsing and compilation

- `_Token` stores a token's kind, text, and source offset; `_Parser` consumes tokens and builds the AST.
- `LambdaSyntaxError` reports malformed source with line and column; `TypeCheckError` reports static type failures.
- `_TypeSolver` creates `TMeta` variables and records substitutions while unifying types.
- `_PyCompiler` walks the expression AST; `_PyComp` carries the generated result name and instruction list.
- Instructions are dataclass variants: `_PyAssign`, `_PyDef`, and `_PyIf`.

These helper classes support separate stages. The expression and type nodes are the shared data passed between parsing, checking, interpreting, and Python emission.

Compilation uses result registers

Each recursive compiler call returns a result name and ordered instruction objects. The emitter turns these objects into Python statements:

```
_PyComp(result="_v6", instructions=[
    _PyAssign(target="_v4", value="1"),
    _PyAssign(target="_v5", value="(_a1 - _v4)"),
    _PyAssign(target="_v6", value="_v1(_v5)"),
])
```

- `reg_new()` creates names such as `_v1`, `_v2`.
- The compile environment maps LAMBDA names to Python names, for example `n -> _a1`.
- `_PyDef` and `_PyIf` contain nested instruction lists, so function and branch bodies stay structured.

Instruction objects and Python emission

Assignment instructions store a target and expression. Control-flow instructions store child blocks and their result registers:

```
_PyIf(test="_v3", result="_v9",
  then_body=[_PyAssign("_v4", "1")], then_result="_v4",
  else_body=[_PyAssign("_v5", "2")], else_result="_v5")
```

- The recursive emitter prints each instruction at the requested indentation.
- `d0exp_compile(ast)` returns the complete generated program.
- `d0exp_emit(ast)` prints that program to standard output.

Scope and evaluation order

The compiler environment maps each LAMBDA name to a generated Python local. Extending a scope copies that map, so shadowing affects only the body.

A let aliases its value register

```
value = compile(rhs, env)
body_env = dict(env)
body_env["x"] = value.result
body = compile(body, body_env)
```

No extra Python assignment is needed. Sequential declarations extend the environment after each initializer.

Instructions preserve call-by-value order

```
function instructions
argument instructions
result = function(argument)
```

Binary operators likewise emit left operand code, then right operand code, followed by the operation.

Recursive functions become Python functions

A LAMBDA `fix fact(n) => body` becomes a Python `def`. The compiler maps the recursive name to the function's own generated name.

```
def _v1(_a1):
    _v2 = 1
    _v3 = (_a1 <= _v2)
    if _v3:
        _v4 = 1
        _v9 = _v4
    else:
        _v5 = 1
        _v6 = (_a1 - _v5)
        _v7 = _v1(_v6)       # recursive call
        _v8 = (_a1 * _v7)
        _v9 = _v8
    return _v9
```

The nested `def` also closes over its surrounding registers.

Conditionals preserve LAMBDA evaluation

The compiler emits branch bodies inside Python's `if/else`. Both branches are compiled, but only the selected branch executes.

```
if _v3:
    _v4 = 1
    _v9 = _v4
else:
    _v5 = 1
    _v6 = (_a1 - _v5)
    _v7 = _v1(_v6)
    _v8 = (_a1 * _v7)
    _v9 = _v8
```

`_v9` is the merge register: each branch assigns it, and the surrounding code uses it as the conditional's result.

Generated module and running the example

The compiler wraps the instruction list in a zero-argument entry point, adds runtime helpers for LAMBDA lists, options, and printing, then invokes the entry point.

```
def _lambda_main():
    # generated instructions
    return _v18

_lambda_result = _lambda_main()
```

From `LAMBDA2/mytest`:

```
make check           # type-check and compare with the interpreter
make show-factorial  # print the generated Python source
```

The factorial example returns `(1, 1, 120, 720)`.

Runtime values and helper calls

Generated code uses Python's built-in values where they fit the LAMBDA data model, and helper functions for operations that need checks or special representations.

- Integers and booleans use Python `int` and `bool`.
- Lists and tuples use Python lists and tuples.
- `list_head`, `list_tail`, and `list_cons` call runtime helpers that check their inputs.
- `optn_nil` is `None`; `optn_cons(x)` is a tagged pair, so an option containing `None` stays distinct.
- Source `/` emits Python `//` for integer division.
- `print` formats the value, prints it, and returns unit (`()`).